



Shellcodes

We should have a very good understanding of the computer and processor architecture and how programs interact with this underlying architecture through what we have learned in this module. We should also be able to disassemble and debug binaries and get a good understanding of what machine instructions they are executing and what their general purpose is. Now we will learn about `shellcodes`, which is an essential concept for penetration testers.

What is a Shellcode

We know that each executable binary is made of machine instructions written in Assembly and then assembled into machine code. A `shellcode` is the hex representation of a binary's executable machine code. For example, let's take our `Hello World` program, which executes the following instructions:

```
global _start

section .data
    message db "Hello HTB Academy!"
```

nasm

```
section .text
_start:
    mov rsi, message
    mov rdi, 1
    mov rdx, 18
    mov rax, 1
    syscall

    mov rax, 60
    mov rdi, 0
    syscall
```

As we have seen in the first section, this `Hello World` program assembles the following shellcode:

```
shellcode
48be0020400000000000bf01000000ba12000000b801000000f05b83c00000bf00000000f05
```

This shellcode should properly represent the machine instructions, and if passed to the processor memory, it should understand it and execute it properly.

Use in Pentesting

Having the ability to pass a shellcode directly to the processor memory and have it executed plays an essential role in `Binary Exploitation`. For example, with a buffer overflow exploit, we can pass a `reverse shell` shellcode, have it executed, and receive a reverse shell.

Modern `x86_64` systems may have protections against loading shellcodes into memory. This is why `x86_64` binary exploitation usually relies on `Return Oriented Programming (ROP)`, which also requires a good understanding of the assembly language and computer architecture covered in this module.

Furthermore, some attack techniques rely on infecting existing executables (like `elf` or `.exe`) or libraries (like `.so` or `.dll`) with shellcode, such that this shellcode is loaded into memory and executed once these files are run. Another advantage of using shellcodes in pentesting is the ability to directly execute code into memory without writing anything to the disk, which is very important for reducing our visibility and footprint on the remote server.

Assembly to Machine Code

To understand how shellcodes are generated, we must first understand how each instruction is converted into a machine code. Each `x86` instruction and each register has its own `binary` machine code (usually represented in `hex`), which represents the binary code passed directly to the processor to tell it what instruction to execute (through the Instruction Cycle.)

Furthermore, common combinations of instructions and registers have their own machine code as well. For example, the `push rax` instruction has the machine code `50`, while `push rbx` has the machine code `53`, and so on. When we assemble our code with `nasm`, it converts our assembly instructions to their respective machine code so that the processor can understand them.

Remember: Assembly language is made for human readability, and the processor cannot understand it without being converted into machine code. We will use `pwntools` to assemble and disassemble our machine code, as it is an essential tool for Binary Exploitation, and this is an

excellent opportunity to start learning it. First, we can install `pwntools` with the following command (it should be already installed in PwnBox):

```
kimvutrinh@htb[/htb]$ sudo pip3 install pwntools
```

shellsession

Now, we can use `pwn asm` to assemble any assembly code into its shellcode, as follows:

```
kimvutrinh@htb[/htb]$ pwn asm 'push rax' -c 'amd64'
0:    50                push    eax
```

shellsession

Note: We used the `-c 'amd64'` flag to ensure the tool properly interprets our assembly code for `x86_64`

As we can see, we get `50`, which is the same machine code for `push rax`. Likewise, we can convert hex machine code or shellcode into its corresponding assembly code, as follows:

```
kimvutrinh@htb[/htb]$ pwn disasm '50' -c 'amd64'
0:    50                push    eax
```

shellsession

We can read more about `pwntools` assembly and disassembly features [here](#), and about the `pwntools` command-line tools [here](#).

Extract Shellcode

Now that we understand how each assembly instruction is converted into machine code (and vice-versa), let's see how to extract the shellcode from any binary.

A binary's shellcode represents its executable `.text` section only, as shellcodes are meant to be directly executable. To extract the `.text` section with `pwntools`, we can use the `ELF` library to load an `elf` binary, which would allow us to run various functions on it. So, let's run the `python3` interpreter to understand better how to use it. First, we'll have to import `pwntools`, and then we can read the `elf` binary, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ python3

>>> from pwn import *
>>> file = ELF('helloworld')
```

Now, we can run various `pwntools` functions on it, which we can read more about [here](#). We need to dump machine code from the executable `.text` section, which we can do with the `section()` function, as follows:

```
shellsession
>>> file.section(".text").hex()
'48be0020400000000000bf01000000ba12000000b801000000f05b83c000000bf000000000f05 '
```

Note: We added ' `hex()` ' to encode the shellcode in hex, instead of printing it in raw bytes.

We see that we were very easily able to extract the binary's shellcode. Let's turn this into a Python script so that we can quickly use it to extract the shellcode of any binary:

```
#!/usr/bin/python3

import sys
from pwn import *

context(os="linux", arch="amd64", log_level="error")

file = ELF(sys.argv[1])
shellcode = file.section(".text")
print(shellcode.hex())
```

We can copy the above script to `shellcoder.py`, and then pass it any binary file's name as an argument, and it'll extract it's shellcode:

```
kimvutrinh@htb[/htb]$ python3 shellcoder.py helloworld

48be0020400000000000bf01000000ba12000000b801000000f05b83c000000bf000000000f05
```

Another (somewhat less reliable) method to extract the shellcode would be through `objdump`, which we've used in a previous section. We can write the following `bash` script into `shellcoder.sh` and use it to extract the shellcode if we can't use the first script:

```
#!/bin/bash

for i in $(objdump -d $1 |grep "^ " |cut -f2); do echo -n $i; done; echo;
```

Again, we can try running it on `helloworld` to get its shellcode, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ ./shellcoder.sh helloworld

48be0020400000000000bf01000000ba12000000b801000000f05b83c000000bf000000000f05
```

Loading Shellcode

Now that we have a shellcode, let's try to run it, allowing us to test any shellcode we have prepared before using it in Binary Exploitation. The shellcode we extracted above does not meet the `Shellcoding Requirements` we'll discuss in the next section, and so it won't run. To demonstrate how to run shellcodes, we'll use the following (`fixed`) shellcode, that meets all `Shellcoding Requirements` :

```
shellcode
4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40b7
◀────────────────────────────────────────────────────────────────────────────────▶▶
```

To run our shellcode with `pwntools` , we can use the `run_shellcode` function and pass it our shellcode, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ python3

>>> from pwn import *
>>> context(os="linux", arch="amd64", log_level="error")
>>>
```

```
run_shellcode(unhex('4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e
```

Hello HTB Academy!

We used `unhex()` on the shellcode to convert it back to binary.

As we can see, our shellcode successfully ran and printed the string `Hello HTB Academy!`. In contrast, if we run the previous shellcode (which did not meet `Shellcoding Requirements`), it will not run:

```
shellsession
>>>
run_shellcode(unhex('b801000000bf0100000048be0020400000000000ba120000000f05b83c000000b
```

Once again, to make it easy to run our shellcodes, let's turn the above into a Python script:

```
python
#!/usr/bin/python3

import sys
from pwn import *

context(os="linux", arch="amd64", log_level="error")

run_shellcode(unhex(sys.argv[1])).interactive()
```

We can copy the above script to `loader.py` , pass our shellcode as an argument, and run it to execute our shellcode:

```
shellsession
kimvutrinh@htb[/htb]$ python3 loader.py
'4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40t

Hello HTB Academy!
```

As we can see, we were able to load and run our shellcode successfully.

Debugging Shellcode

Finally, let's see how we can debug our shellcode with `gdb` . If we are loading the machine code directly into memory, how would we run it with `gdb` ? There are many ways to do so, and we'll go through some of them here.

We can always run our shellcode with `loader.py` , and then attach its process to `gdb` with `gdb -p PID` . However, this will only work if our process does not exit before we attach to it. So, we will instead build our shellcode to an `elf` binary and then use this binary with `gdb` like we've been doing throughout the module.

Pwntools

We can use `pwntools` to build an `elf` binary from our shellcode using the `ELF` library, and then the `save` function to save it to a file:

```
python
```

```
ELF.from_bytes(unhex('4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40t
```

To make it easier to use, we can turn the above into a script and write it to `assembler.py`:

```
#!/usr/bin/python3

import sys, os, stat
from pwn import *

context(os="linux", arch="amd64", log_level="error")

ELF.from_bytes(unhex(sys.argv[1])).save(sys.argv[2])
os.chmod(sys.argv[2], stat.S_IEXEC)
```

We can now run `assembler.py`, pass the shellcode as the first argument, and the file name as the second argument, and it'll assemble the shellcode into an executable:

```
kimvutrinh@htb[/htb]$ python assembler.py
'4831db66bb79215348bb422041636164656d5348bb48656c6c6f204854534889e64831c0b0014831ff40t
'helloworld'
```

```
kimvutrinh@htb[/htb]$ ./helloworld
```

```
Hello HTB Academy!
```

As we can see, it built the `helloworld` binary with the file name we specified. We can now run it with `gdb`, and use `b *0x401000` to break at the default binary entry point:

```
$ gdb -q helloworld
gef> b *0x401000
gef> r
Breakpoint 1, 0x0000000000401000 in ?? ()
...SNIP...

code:x86:64  —●→  0x401000      xor    rbx, rbx
               0x401003      mov    bx, 0x2179
               0x401007      push   rbx
```

GCC

There are other methods to build our shellcode into an `elf` executable. We can add our shellcode to the following `C` code, write it to a `helloworld.c`, and then build it with `gcc` (hex bytes must be escaped with `\x`):

```
#include <stdio.h>

int main()
{
    int (*ret)() = (int (*)())
"\x48\x31\xdb\x66\xbb\...\SNIP...\x3c\x40\x30\xff\x0f\x05";
    ret();
}
```

Then, we can compile our `C` code with `gcc`, and run it with `gdb`:

```
shellsession
kimvutrinh@htb[/htb]$ gcc helloworld.c -o helloworld
kimvutrinh@htb[/htb]$ gdb -q helloworld
```

However, this method is not very reliable for a few reasons. First, it will wrap the entire binary in `C` code, so the binary will not contain our shellcode, but will contain various other `C` functions and libraries. This method may also not always compile, depending on the existing memory protections, so we may have to add flags to bypass memory protections, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ gcc helloworld.c -o helloworld -fno-stack-protector -z
execstack -Wl,--omagic -g --static
kimvutrinh@htb[/htb]$ ./helloworld

Hello HTB Academy!
```

With this, we should have a good understanding of the basics of shellcodes. We can now create our own shellcodes for our next steps.

Exercise Shellcode

```
4831db536a0a48b86d336d307279217d5048b833645f316e37305f5048b84854427b6c3034
64504889e64831c0b0014831ff40b7014831d2b2190f054831c0043c4030ff0f05
```

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

+1

+40



Run the "Exercise Shellcode" to get the flag.

Write your answer

Submit

← Previous

Section 21 / 24

+20

Next

Mark Complete & Next